Hardening d’un serveur Linux (BIOS, GRUB, chiffrement des données)

Nom Prénom : **Dylan Spinelli**

BTS SIO SISR

Paris Ynov Campus

Sommaire

1- Faire une installation avec un disque LVM chiffré..... 1

2- Sécuriser le BIOS..... 1

3- Sécuriser le GRUB (GRand Unified Bootloader)..... 2

a- Présentation..... 2

b- Détecter la version du GRUB..... 3

c- Mise en place d'un mot de passe chiffré sur le GRUB..... 3

d- Conserver le paramétrage en cas de mise à jour..... 4

4- Chiffrer les données..... 4

a- Introduction..... 4

b- Création d'une partition chiffrée avec cryptsetup..... 4

c- Ouvrons le conteneur chiffré..... 5

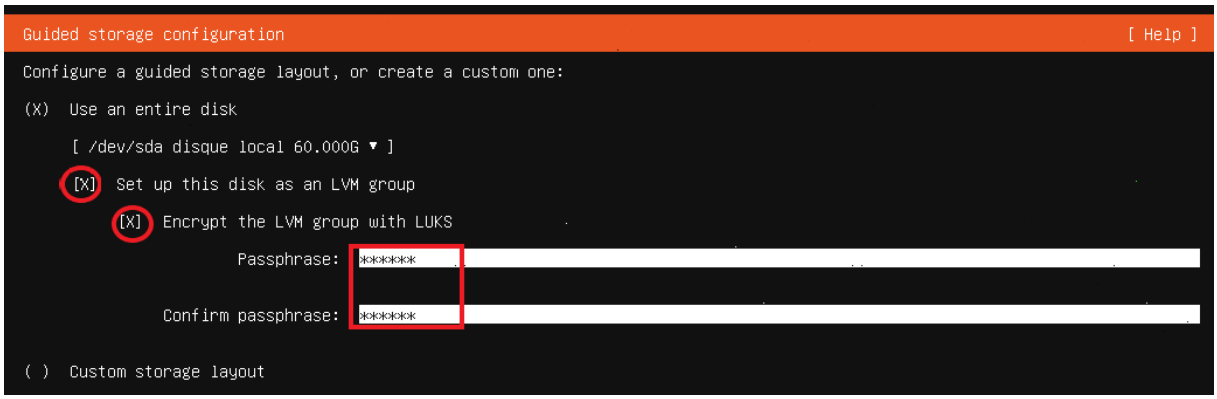
d- Création d'un système de fichiers ext4..... 5

e- Ajouter une phrase de passe supplémentaire..... 5

f- Démonter le disque..... 6

g- Test de notre disque chiffré..... 6

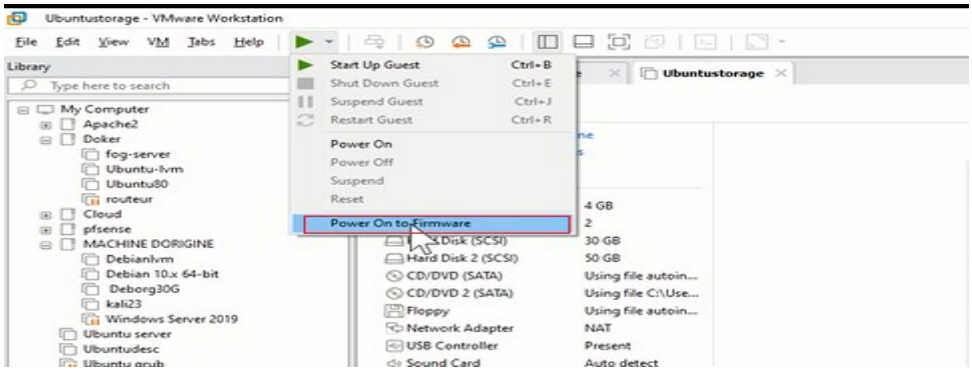
1- Faire une installation avec un disque LVM chiffré

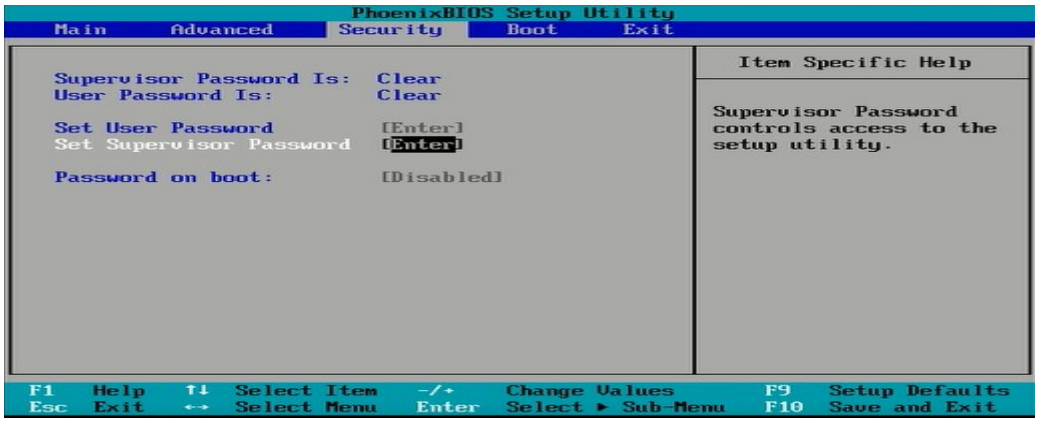


2- Sécuriser le BIOS

Protégez le BIOS par mot de passe, même si le mot de passe est volatile suite à l'enlèvement de la pile
Désactivez le boot via CD/DVD, USB, Périphérique externe, car le Boot live permet à quiconque ayant un accès physique à la machine d'accéder au système.

Pour accéder au bios sur VMWare il faut démarrez votre machine en cliquant sur **Power On to Firmware**





3- Sécuriser le GRUB (GRand Unified Bootloader)

a- Présentation

GRUB est le chargeur de démarrage Linux, en démarrant en mode rescue quiconque ayant un accès physique à la machine peut modifier le mot de passe root.

Mode de dépannage (recovery mode) :

Le **mode de dépannage** ou **mode de récupération** (en anglais, *recovery mode*) est une méthode de démarrage d'Ubuntu vous permettant d'effectuer certaines tâches d'administration et de récupération de votre système Ubuntu. Ce mode est très utile lorsque :

- Aucune session n'est en mesure d'être ouverte car vos disques durs sont remplis,
- Les systèmes de fichiers sont corrompus et que le démarrage d'Ubuntu est impossible,
- Lorsqu'une session graphique ne peut pas être démarrée (ex : une mise à jour du serveur graphique ne gère plus vos anciens pilotes propriétaires et bloque le démarrage d'une session) ou
- Lorsque le mot de passe du compte d'utilisateur principal est oublié…

Faille de sécurité

le mode de dépannage charge le système Ubuntu directement en session super-utilisateur (*root*) sans fournir un mot de passe. Ceci représente une certaine vulnérabilité pour votre système, car tout utilisateur ayant un accès physique à votre ordinateur peut démarrer en mode de secours. Ceci est un risque de deux manières :

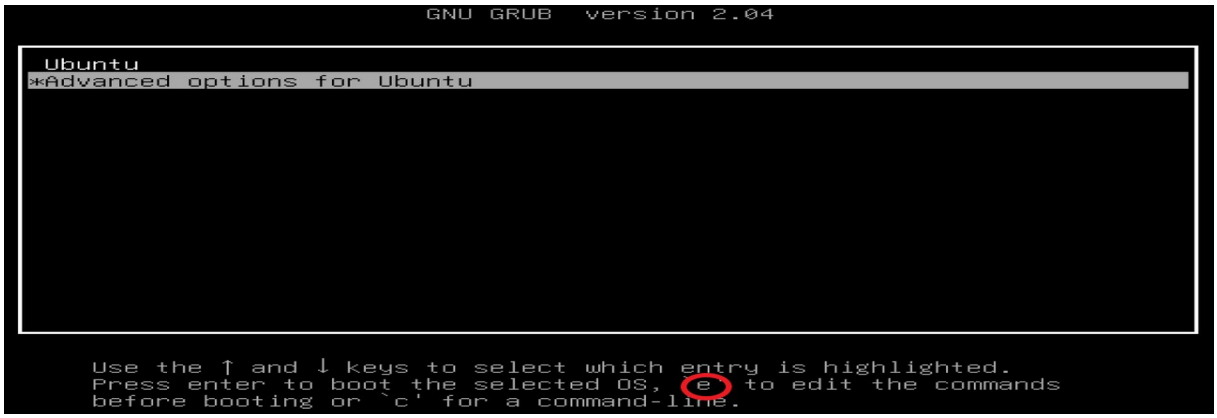
a- Puisque tout utilisateur ayant un accès physique à votre ordinateur peut démarrer en mode de dépannage, un utilisateur malintentionné de votre machine peut corrompre votre système Ubuntu sans aucune difficulté ;

b- Puisqu'une session en mode de dépannage est automatiquement ouverte avec le compte super-utilisateur, une action malencontreuse faite par vous, de manière accidentelle, peut corrompre complètement votre système Ubuntu.

Démarrer le **mode de dépannage**

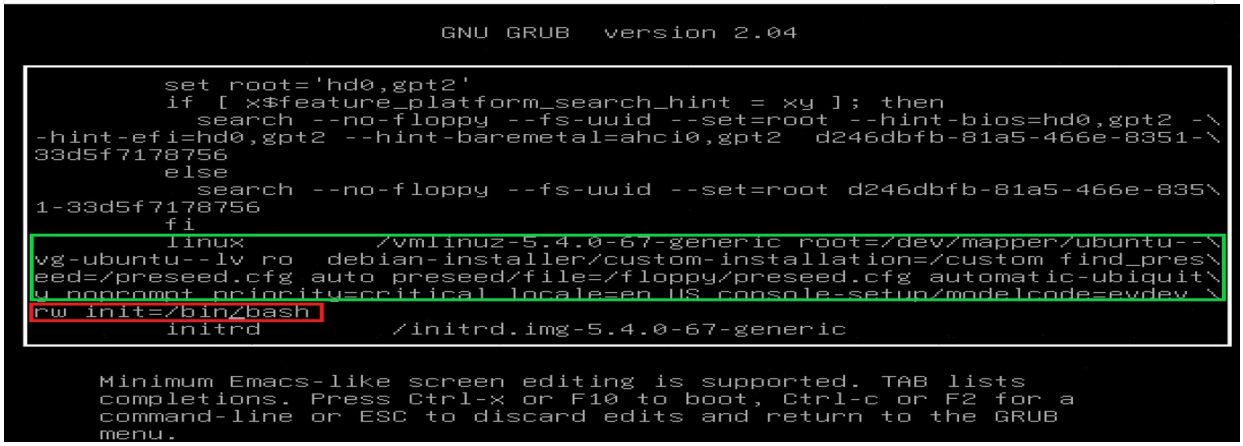
Pour forcer l'affichage de ce menu lorsque votre système **Ubuntu**, appuyez sur la touche [Échap] (ou [ESC]) ou sur la touche [⇧ Maj] (ou [⇧ Shift]) et maintenez-la enfoncée jusqu'à ce que le menu d'amorçage s'affiche.

Dès que le menu Grub apparait appuyer sur **e** pour éditer ce menu



Cherchez la ligne qui commence par *linux* (l'encadré en rouge) et ajoutez à la fin de cette ligne **rw init=/bin/bash** (l'encadré en rouge)

Puis faites un F10 ou CTRL X pour booter. On va tomber sur un shell bash en root.

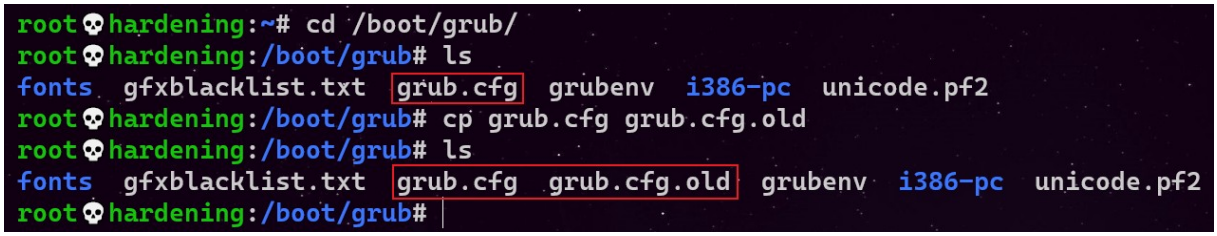


b- Détecter la version du GRUB



c- Mise en place d'un mot de passe chiffré sur le GRUB

Faire une sauvegarde du fichier de configuration par sécurité



grub-mkpasswd-pbkdf2

La commande va alors demander de saisir le mot de passe (deux fois), puis va retourner le hash de celui-ci (en SHA1 sur 512bits).



Ensuite il faut éditer le fichier grub.cfg et y coller le hash du mot de passe :

```
#password grub
set superusers="root"
password_pbkdf2 root
grub.pbkdf2.sha512.10000.DCDF1C480A8710D47E6910BFAB847D6A819E10994D914DDBA3B0D1DAE0158A1B62703C1D27
FAFBEEA117A1B80E2D3B041B9180A15F30745D625BF11AFC9AA6E5.7D6FED39FEEA81686C06CBE63FB6DF7DABCC14F1F4
6A42C811BC1FFB9BFF8F4AEC4E1B026E55CAF8F0F0CF85F74DF706B2FCAABA7595D95FD16ED300FDE80A4B
```

d- Conserver le paramétrage en cas de mise à jour

En cas de mise à jour du menu, votre paramétrage disparaîtra. Pour que la modification perdure après les mises à jour, il faut d'éditer le fichier /etc/grub.d/00_header

```
root💀ubuntulvm:~# vim /etc/grub.d/00_header
```

Pour y ajouter :

```
cat << EOF
set superusers="root"
password_pbkdf2 root
grub.pbkdf2.sha512.10000.DCDF1C480A8710D47E6910BFAB847D6A819E10994D914DDBA3B0D1DAE0158A1B
62703C1D27FAFBEEA117A1B80E2D3B041B9180A15F30745D625BF11AFC9AA6E5.7D6FED39FEEA81686C06CBE
63FB6DF7DABCC14F1F46A42C811BC1FFB9BFF8F4AEC4E1B026E55CAF8F0F0CF85F74DF706B2FCAABA7595D95
FD16ED300FDE80A4B
EOF
```

4- Chiffrer les données

a- Introduction



Dans cette partie, nous allons voir comment créer une partition chiffrée avec cryptsetup. Le but est de chiffrer une partition d'un disque interne ou externe



Tout d'abord il faut vérifier que le paquet cryptsetup est installée, si ce n'est pas le cas il faut bien sûre l'installer

```
root💀debian:/# apt install cryptsetup
```

b- Création d'une partition chiffrée avec cryptsetup

La syntaxe est la suivante

```
#cryptsetup --verify-passphrase --cipher= --key-size
```

Exemple

```
#cryptsetup --verify-passphrase --cipher serpent-cbc-essiv:sha256 --key-size 256 luksFormat
/dev/sdb1
```

On fait simple on va laisser le système déterminer les options par défaut
On va créer le conteneur chiffré avec la commande suivante :

```
root💀storage:~# cryptsetup --verify-passphrase luksFormat /dev/sdb1
ATTENTION: Le périphérique /dev/sdb1 contient déjà une signature pour un superblock « ext4 ».

WARNING!
========
Cette action écrasera définitivement les données sur /dev/sdb1.

Are you sure? (Type uppercase yes): YES
Saisissez la phrase secrète pour /dev/sdb1 :
Vérifiez la phrase secrète :
```

Attention YES en majuscule

c- Ouvrons le conteneur chiffré

On va ouvrir le conteneur avec luksOpen. On va appeler notre conteneur **sdb1crypt** (on peut mettre le nom qu'on veut) :

```
root💀storage:~# cryptsetup luksOpen /dev/sdb1 sdb1crypt
Enter passphrase for /dev/sdb1:
```

d- Création d'un système de fichiers ext4

#fdisk -l pour vérification

```
Disque /dev/mapper/sdb1crypt : 10 GiB, 10720641024 octets, 20938752 secteurs
Unités : secteur de 1 × 512 = 512 octets
Taille de secteur (logique / physique) : 512 octets / 512 octets
taille d'E/S (minimale / optimale) : 512 octets / 512 octets
```

i- Formater la partition

```
root💀storage:~# mkfs -t ext4 /dev/mapper/sdb1crypt
mke2fs 1.45.5 (07-Jan-2020)
Creating filesystem with 2617344 4k blocks and 655360 inodes
Filesystem UUID: 26ff15a3-6979-4683-b2dd-bd40d8405914
Superblock backups stored on blocks:
        32768, 98304, 163840, 229376, 294912, 819200, 884736, 1605632

Allocating group tables: done
Writing inode tables: done
Creating journal (16384 blocks): done
Writing superblocks and filesystem accounting information: done
```

ii- Monter la partition

```
root💀storage:~# mount /dev/mapper/sdb1crypt /mnt
```

On peut mettre des données dans notre partition sdb1crypt

```
root💀storage:/mnt# mkdir data
root💀storage:/mnt# touch file.txt
root💀storage:/mnt# ls
data  file.txt
root💀storage:/mnt#
```

e- Ajouter une phrase de passe supplémentaire

On peut rajouter jusqu'à 8 clés si on veut que chaque individue ait sa propre clé

```
root💀storage:~# cryptsetup luksAddKey /dev/sdb1
Enter any existing passphrase:
Enter new passphrase for key slot:
Verify passphrase:
```

On peut vérifier le nombre des clés avec la commande suivante

```
root💀storage:~# cryptsetup luksDump /dev/sdb1
```

Les deux clés créées sont encadrées en vert

```
Keyslots:
  0: luks2
        Key:        512 bits
        Priority:   normal
        Cipher:     aes-xts-plain64
        Cipher key: 512 bits
        PBKDF:      argon2i
        Time cost:  4
        Memory:     311693
        Threads:    2
        Salt:       7b ee 30 1e f6 31 69 60 d6 24 fe 92 54 ed 5a 1d
                    aa d6 e9 c0 92 35 0d 27 f0 18 60 b0 e7 5f 09 c0
        AF stripes: 4000
        AF hash:    sha256
        Area offset:32768 [bytes]
        Area length:258048 [bytes]
        Digest ID:  0
  1: luks2
        Key:        512 bits
        Priority:   normal
        Cipher:     aes-xts-plain64
        Cipher key: 512 bits
        PBKDF:      argon2i
        Time cost:  4
        Memory:     339122
        Threads:    2
        Salt:       19 f2 0a a1 cb 69 12 d6 7c e9 2c 61 3e 14 df 40
                    d6 dd 76 c8 c8 72 42 bb ed 65 a5 0d a2 90 7c f3
        AF stripes: 4000
        AF hash:    sha256
        Area offset:290816 [bytes]
        Area length:258048 [bytes]
        Digest ID:  0
```

Pour supprimer une clé c'est simple on utilise la même commande sauf que on remplace **Add** par **Remove** et on entre le mot de passe à supprimer

```
root💀storage:~# cryptsetup luksRemoveKey /dev/sdb1
Enter passphrase to be deleted:
```

f- Démonter le disque

Maintenant on va démonter notre disque crypté sdb1 pour déterminer son point de montage on utilise la command df qui répertorie tous les points de montage

```
root💀storage:~# df
Filesystem                         1K-blocks    Used Available Use% Mounted on
udev                                 1956456       0   1956456   0% /dev
tmpfs                                 400172    1560    398612   1% /run
/dev/mapper/ubuntu--vg-ubuntu--lv   20511312 6458096  12988256  34% /
tmpfs                                2000852       0   2000852   0% /dev/shm
tmpfs                                   5120       0      5120   0% /run/lock
tmpfs                                2000852       0   2000852   0% /sys/fs/cgroup
/dev/sda2                             999320  107020    823488  12% /boot
/dev/loop0                             56832   56832         0 100% /snap/core18/1944
/dev/loop1                             71680   71680         0 100% /snap/lxd/19188
/dev/loop2                             56832   56832         0 100% /snap/core18/1997
/dev/loop3                             33152   33152         0 100% /snap/snapd/11588
/dev/loop4                             31872   31872         0 100% /snap/snapd/10707
/dev/loop5                             69248   69248         0 100% /snap/lxd/20326
tmpfs                                 400168       0    400168   0% /run/user/1000
/dev/mapper/sdb1crypt               10239252   36888   9662512   1% /mnt
```

Notre point de montage est encadré en vert on utilise umount pour démonter le disque

```
root💀storage:~# umount /mnt
```

Maintenant il faut fermer notre conteneur chiffré

```
root💀storage:~# cryptsetup luksClose sdb1crypt
```

g- Test de notre disque chiffré

Il faut toujours faire un fdisk -l pour localiser notre disque car le nom peut changer en fonction d'un nouveau démarrage

```
root💀storage:~# fdisk -l
```

```
Disk /dev/sdb: 50 GiB, 53687091200 bytes, 104857600 sectors
Disk model: VMware Virtual S
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
Disk identifier: 0x1e235241

Device     Boot Start      End  Sectors Size Id Type
/dev/sdb1        2048 20973567 20971520  10G 83 Linux
```

Après on ouvre le conteneur chiffré on peut changer le nom de sdb1crypte à datacrypte
On rentre le mot de passe qu'on a entré précédemment

```
root💀storage:~# cryptsetup luksOpen /dev/sdb1 datacrypt
Enter passphrase for /dev/sdb1:
root💀storage:~#
```

On vérifie avec fdisk -l pour identifier le volume à monter

```
root💀storage:~# fdisk -l
Disk /dev/mapper/datacrypt: 9.101 GiB, 10720641024 bytes, 20938752 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
```

On monte notre volume **/dev/mapper/datacrypt**

```
root💀storage:~# mount /dev/mapper/datacrypt /mnt
```

On vérifie avec df que le montage a réussi

```
root💀storage:~# df
Filesystem                         1K-blocks    Used Available Use% Mounted on
udev                                 1956456       0   1956456   0% /dev
tmpfs                                 400172    1560    398612   1% /run
/dev/mapper/ubuntu--vg-ubuntu--lv   20511312 6458360  12987992  34% /
tmpfs                                2000852       0   2000852   0% /dev/shm
tmpfs                                   5120       0      5120   0% /run/lock
tmpfs                                2000852       0   2000852   0% /sys/fs/cgroup
/dev/sda2                             999320  107020    823488  12% /boot
/dev/loop0                             56832   56832         0 100% /snap/core18/1944
/dev/loop1                             71680   71680         0 100% /snap/lxd/19188
/dev/loop2                             56832   56832         0 100% /snap/core18/1997
/dev/loop3                             33152   33152         0 100% /snap/snapd/11588
/dev/loop4                             31872   31872         0 100% /snap/snapd/10707
/dev/loop5                             69248   69248         0 100% /snap/lxd/20326
tmpfs                                 400168       0    400168   0% /run/user/1000
/dev/mapper/datacrypt               10239252   36888   9662512   1% /mnt
```

Si on fait un ls on retrouve nos donnés

```
root💀storage:/crypte# ls
data  file.txt  lost+found
```